

Strings em Java: Trabalhando com Textos

O que é uma String?

No dia a dia, programas precisam trabalhar com nomes, mensagens, senhas, cidades, e-mails, palavras e frases. Em Java, usamos a classe **String** para guardar esse tipo de informação.

Definição: String

Uma **String** representa um texto, isto é, uma sequência de caracteres. Esse texto pode conter letras, números, espaços e símbolos.

```
1 String nome = "Maria";  
2 String cidade = "Patos";  
3 String mensagem = "Ola, mundo!";  
4 String codigo = "A123";
```

Listing 1: Criando variáveis do tipo String

Importante

Mesmo quando uma String contém apenas números, ela continua sendo **texto**. Por exemplo, "123" é uma String, enquanto 123 é um número inteiro.

String não é um tipo primitivo

Em Java, tipos como `int`, `double`, `char` e `boolean` são tipos primitivos. Já **String** é uma **classe**, e uma variável desse tipo trabalha com um objeto que representa texto.

Uma forma simples de perceber a diferença

Um número inteiro pode ser usado diretamente em operações matemáticas. Uma String, por ser um objeto, oferece vários **métodos prontos** para trabalhar com o texto, como descobrir seu tamanho, transformar letras em maiúsculas ou procurar um trecho.

```
1 String palavra = "casa";  
2  
3 System.out.println(palavra.length());
```

```

4 System.out.println(palavra.toUpperCase());
5 System.out.println(palavra.charAt(0));

```

Listing 2: Uma String possui métodos prontos

A saída será:

```

1 4
2 CASA
3 c

```

Em Java, Strings são objetos e possuem métodos que ajudam a manipular textos.

Criando e mostrando Strings

A forma mais comum de criar uma String é colocar o texto entre **aspas duplas**.

```

1 String nome = "Ana";
2
3 System.out.println(nome);

```

Listing 3: Criando e mostrando uma String

Atenção às aspas

Aspas duplas representam uma **String**. Aspas simples representam um único caractere do tipo **char**.

```

1 String palavra = "A";
2 char letra = 'A';

```

Listing 4: String e char são diferentes

Juntando textos: concatenação

Podemos juntar Strings utilizando o operador **+**. Esse processo é chamado de **concatenação**.

```

1 String nome = "Carlos";
2 int idade = 20;
3
4 System.out.println("Nome: " + nome);
5 System.out.println("Idade: " + idade);

```

Listing 5: Concatenando textos

Também podemos montar uma nova String aos poucos:

```

1 String resultado = "";
2
3 resultado += "c";
4 resultado += "a";
5 resultado += "s";
6 resultado += "a";

```

```

7
8 System.out.println(resultado); // casa

```

Listing 6: Construindo uma String com +=

String vazia

A escrita "" representa uma String sem nenhum caractere. Ela pode ser útil quando queremos construir um texto pouco a pouco.

Tamanho e posições de uma String

Cada caractere de uma String ocupa uma posição. Em Java, a contagem das posições começa em zero.

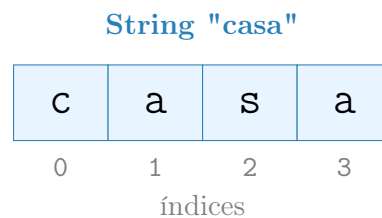


Figure 1: As posições de uma String começam em zero.

Método length()

O método length() informa quantos caracteres existem na String.

```

1 String palavra = "casa";
2
3 System.out.println(palavra.length()); // 4

```

Listing 7: Descobrimos o tamanho de uma String

String e vetor usam escritas diferentes

Para String usamos length(). Para vetor usamos length, sem parênteses.

Acessando um caractere com charAt()

O método charAt(indice) devolve o caractere existente em uma posição.

```

1 String palavra = "casa";
2
3 System.out.println(palavra.charAt(0)); // c
4 System.out.println(palavra.charAt(1)); // a
5 System.out.println(palavra.charAt(2)); // s
6 System.out.println(palavra.charAt(3)); // a

```

Listing 8: Acessando caracteres por posição

Índice fora do limite

Se a String "casa" possui 4 caracteres, seus índices vão de 0 até 3. Tentar usar `charAt(4)` provoca erro durante a execução.

Percorrendo uma String

Como cada caractere possui um índice, podemos usar um laço para visitar todos os caracteres da String.

```
1 String palavra = "casa";
2
3 for (int i = 0; i < palavra.length(); i++) {
4     char letra = palavra.charAt(i);
5     System.out.println(letra);
6 }
```

Listing 9: Percorrendo uma String com for

A saída será:

```
1 c
2 a
3 s
4 a
```

Como pensar nesse laço?

A variável `i` representa a posição atual. Primeiro acessamos a posição 0, depois 1, depois 2, e seguimos até a última posição existente.

Transformando em vetor de char

O método `toCharArray()` transforma uma String em um vetor de caracteres.

```
1 String palavra = "casa";
2 char[] letras = palavra.toCharArray();
3
4 for (int i = 0; i < letras.length; i++) {
5     System.out.println(letras[i]);
6 }
```

Listing 10: Convertendo String para char[]

Quando usar?

Para muitos problemas simples, `charAt()` já é suficiente. O `toCharArray()` é apenas outra forma de trabalhar caractere por caractere.

Comparando Strings

Quando queremos saber se duas Strings possuem o mesmo texto, usamos o método `equals()`.

```

1 String palavra1 = "casa";
2 String palavra2 = "casa";
3
4 if (palavra1.equals(palavra2)) {
5     System.out.println("As palavras sao iguais.");
6 }

```

Listing 11: Comparando textos com equals()

Não use == para comparar o conteúdo de Strings

O operador == não é a forma correta de verificar se duas Strings possuem o mesmo texto. Para comparar o conteúdo, use equals().

```

1 String palavra1 = new String("casa");
2 String palavra2 = new String("casa");
3
4 System.out.println(palavra1 == palavra2);           // false
5 System.out.println(palavra1.equals(palavra2));      // true

```

Listing 12: Exemplo que mostra a diferença

Ignorando maiúsculas e minúsculas

O método equalsIgnoreCase() compara dois textos ignorando a diferença entre letras maiúsculas e minúsculas.

```

1 String opcao = "fim";
2
3 if (opcao.equalsIgnoreCase("FIM")) {
4     System.out.println("Encerrando...");
5 }

```

Listing 13: Comparando sem diferenciar maiúsculas e minúsculas

Resumo da comparação

- equals() considera maiúsculas e minúsculas;
- equalsIgnoreCase() ignora essa diferença;
- == não deve ser usado para comparar o conteúdo textual.

Maiúsculas e minúsculas

Dois métodos muito usados são toUpperCase() e toLowerCase().

```

1 String palavra = "Java";
2
3 System.out.println(palavra.toUpperCase()); // JAVA
4 System.out.println(palavra.toLowerCase()); // java

```

Listing 14: Convertendo o texto

Esses métodos são úteis quando queremos padronizar um texto antes de compará-lo.

```
1 String resposta = "Sim";
2
3 resposta = resposta.toLowerCase();
4
5 if (resposta.equals("sim")) {
6     System.out.println("Resposta positiva.");
7 }
```

Listing 15: Padronizando antes de verificar

Strings são imutáveis

Uma característica importante de String em Java é a **imutabilidade**. Isso significa que os métodos não alteram o texto original diretamente: eles produzem outro resultado.

```
1 String palavra = "CASA";
2
3 palavra.toLowerCase();
4
5 System.out.println(palavra); // CASA
```

Listing 16: O texto original não é alterado automaticamente

Para guardar o resultado, precisamos atribuí-lo novamente:

```
1 String palavra = "CASA";
2
3 palavra = palavra.toLowerCase();
4
5 System.out.println(palavra); // casa
```

Listing 17: Guardando a nova String



O método produz um novo texto; o original não é modificado.

Figure 2: Uma operação sobre String produz um novo resultado.

Exemplo simples de imutabilidade

Se duas variáveis apontam inicialmente para o mesmo texto e depois uma delas recebe o resultado de `toUpperCase()`, a outra continua com o texto anterior.

```
1 String s1 = "abc";
2 String s2 = s1;
3
4 System.out.println(s1); // abc
5 System.out.println(s2); // abc
6
```

```

7 s1 = s1.toUpperCase();
8
9 System.out.println(s1); // ABC
10 System.out.println(s2); // abc

```

Listing 18: Uma variável pode receber um novo texto sem alterar a outra

Procurando conteúdo dentro de uma String

Java possui métodos prontos para descobrir se um texto contém determinado trecho ou em qual posição ele aparece.

contains()

O método `contains()` verifica se um trecho existe dentro da String.

```

1 String palavra = "computador";
2
3 System.out.println(palavra.contains("dor")); // true
4 System.out.println(palavra.contains("casa")); // false

```

Listing 19: Verificando se um texto contém outro

startsWith() e endsWith()

Esses métodos verificam como o texto começa ou termina.

```

1 String palavra = "casa";
2
3 System.out.println(palavra.startsWith("ca")); // true
4 System.out.println(palavra.endsWith("sa")); // true

```

Listing 20: Verificando início e fim

indexOf() e lastIndexOf()

O método `indexOf()` devolve a posição da **primeira ocorrência**. Já `lastIndexOf()` devolve a posição da **última ocorrência**.

```

1 String palavra = "banana";
2
3 System.out.println(palavra.indexOf('a')); // 1
4 System.out.println(palavra.indexOf("na")); // 2
5 System.out.println(palavra.lastIndexOf('a')); // 5
6 System.out.println(palavra.indexOf('x')); // -1

```

Listing 21: Localizando caracteres e trechos

O que significa -1?

Quando `indexOf()` não encontra o caractere ou trecho procurado, ele retorna -1.

Extraindo partes de uma String

O método `substring()` permite obter apenas uma parte de um texto.

```
1 String palavra = "computador";
2
3 String parte1 = palavra.substring(0, 4);
4 String parte2 = palavra.substring(4);
5
6 System.out.println(parte1); // comp
7 System.out.println(parte2); // utador
```

Listing 22: Usando `substring()`

Como funciona `substring(inicio, fim)`?

O índice inicial é incluído. O índice final **não é incluído**. Assim, `substring(0, 4)` pega as posições 0, 1, 2 e 3.

`substring(0, 4) = "comp"`

c	o	m	p	u	t	a	d	o	r
0	1	2	3	4	5	6	7	8	9

Figure 3: O índice final da `substring` não faz parte do resultado.

Substituindo partes do texto

O método `replace()` substitui caracteres ou trechos de uma String.

```
1 String palavra = "banana";
2 String novaPalavra = palavra.replace('a', 'o');
3
4 System.out.println(novaPalavra); // bonono
```

Listing 23: Substituindo caracteres

Também podemos substituir um trecho inteiro:

```
1 String frase = "Eu gosto de Java";
2 String novaFrase = frase.replace("Java", "programacao");
3
4 System.out.println(novaFrase);
```

Listing 24: Substituindo trechos

Lembre da imutabilidade

`replace()` não altera a String original. Para guardar a mudança, armazene o resultado em uma variável ou atribua-o novamente à variável existente.

Removendo espaços das extremidades

O método `trim()` remove espaços existentes no começo e no final da String.

```
1 String nome = "   Maria   ";
2
3 nome = nome.trim();
4
5 System.out.println(nome); // Maria
```

Listing 25: Usando `trim()`

O que `trim()` não faz?

Ele não remove os espaços que existem **entre** as palavras. Por exemplo, "Ana Maria" continua contendo o espaço entre Ana e Maria.

Verificando se uma String está vazia

O método `isEmpty()` informa se a String possui zero caracteres.

```
1 String palavra = "";
2
3 if (palavra.isEmpty()) {
4     System.out.println("A String esta vazia.");
5 }
```

Listing 26: Verificando uma String vazia

String vazia não é a mesma coisa que espaço

" " tem zero caracteres. Já " " possui um caractere: um espaço.

Lendo Strings com Scanner

Para receber textos digitados pelo usuário, podemos utilizar a classe `Scanner`.

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Digite uma palavra: ");
8         String palavra = sc.next();
9
10        System.out.println("Voce digitou: " + palavra);
11    }
12 }
```

Listing 27: Lendo uma palavra

O método `next()` lê apenas até o próximo espaço. Para ler uma linha inteira, usamos `nextLine()`.

```
1 System.out.print("Digite uma frase: ");
2 String frase = sc.nextLine();
```

Listing 28: Lendo uma frase inteira

`next()` ou `nextLine()`?

- `next()` é adequado para uma única palavra;
- `nextLine()` é adequado quando a entrada pode conter espaços.

O cuidado ao misturar `nextInt()` e `nextLine()`

Ao usar `nextInt()`, o Enter digitado pelo usuário pode permanecer pendente. Se chamarmos `nextLine()` logo depois, ele pode consumir esse Enter e parecer que “pulou” a leitura.

```
1 System.out.print("Digite a idade: ");
2 int idade = sc.nextInt();
3
4 System.out.print("Digite o nome completo: ");
5 String nome = sc.nextLine();
```

Listing 29: Situação que pode causar leitura inesperada

Uma solução simples é consumir o Enter antes de ler a frase:

```
1 System.out.print("Digite a idade: ");
2 int idade = sc.nextInt();
3 sc.nextLine();
4
5 System.out.print("Digite o nome completo: ");
6 String nome = sc.nextLine();
```

Listing 30: Consumindo o Enter antes de `nextLine()`

Como pensar nesse problema?

O `nextInt()` pega o número, mas não pega o Enter. O primeiro `nextLine()` seguinte encontra esse Enter imediatamente. Por isso usamos uma leitura extra para descartá-lo.

Métodos úteis da classe `String`

A tabela a seguir resume os métodos apresentados neste material.

Método	Para que serve
<code>length()</code>	Retorna a quantidade de caracteres.
<code>charAt(indice)</code>	Retorna o caractere de uma posição.
<code>equals(texto)</code>	Compara o conteúdo de duas Strings.
<code>equalsIgnoreCase(texto)</code>	Compara ignorando maiúsculas e minúsculas.
<code>toUpperCase()</code>	Produz uma versão em letras maiúsculas.
<code>toLowerCase()</code>	Produz uma versão em letras minúsculas.
<code>contains(texto)</code>	Verifica se um trecho existe.
<code>startsWith(texto)</code>	Verifica como a String começa.
<code>endsWith(texto)</code>	Verifica como a String termina.
<code>indexOf(valor)</code>	Retorna a primeira posição encontrada.
<code>lastIndexOf(valor)</code>	Retorna a última posição encontrada.
<code>substring(...)</code>	Extraí uma parte da String.
<code>replace(...)</code>	Substitui caracteres ou trechos.
<code>trim()</code>	Remove espaços do início e do fim.
<code>isEmpty()</code>	Verifica se a String possui zero caracteres.
<code>toCharArray()</code>	Converte a String para um vetor de <code>char</code> .

Métodos da classe Character

Quando acessamos uma posição da String com `charAt()`, recebemos um valor do tipo `char`. A classe `Character` possui métodos úteis para trabalhar com esse caractere.

```

1 char letra = 'A';
2
3 letra = Character.toLowerCase(letra);
4 System.out.println(letra); // a

```

Listing 31: Convertendo um caractere

```

1 char caractere = 'a';
2
3 if (Character.isLetter(caractere)) {
4     System.out.println("É uma letra.");
5 }

```

Listing 32: Verificando se um caractere é uma letra

String e char trabalhando juntos

É muito comum percorrer uma String com `charAt()` e usar métodos de `Character` para analisar cada caractere individualmente.

Exemplo reunindo os conceitos principais

O programa abaixo lê uma palavra e apresenta algumas informações simples sobre ela.

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.print("Digite uma palavra: ");
8         String palavra = sc.next();
9
10        System.out.println("Palavra: " + palavra);
11        System.out.println("Tamanho: " + palavra.length());
12        System.out.println("Primeira letra: " + palavra.charAt(0));
13        System.out.println("Maiuscula: " + palavra.toUpperCase());
14        System.out.println("Minuscula: " + palavra.toLowerCase());
15    }
16 }
```

Listing 33: Exemplo completo e introdutório

O que esse exemplo mostra?

Uma `String` pode ser lida, exibida e manipulada por meio de métodos. Não precisamos criar manualmente operações básicas como descobrir o tamanho ou converter o texto para maiúsculas: a própria classe `String` já oferece essas funcionalidades.

Erros comuns ao trabalhar com Strings

1. Usar `==` para comparar textos

Prefira `equals()` ou `equalsIgnoreCase()`.

2. Esquecer que o primeiro índice é 0

Se uma `String` possui tamanho 4, os índices válidos são 0, 1, 2 e 3.

3. Esquecer de guardar o resultado de um método

Métodos como `toLowerCase()`, `toUpperCase()`, `replace()` e `trim()` produzem uma nova `String`. Se você quiser manter a alteração, precisa guardar o resultado.

4. Confundir `length` com `length()`

`String` usa `length()`. Vetor usa `length`.

5. Misturar `nextInt()` e `nextLine()` sem tratar o Enter

Após uma leitura numérica, pode ser necessário chamar `nextLine()` uma vez antes de ler uma frase.

Resumo final

O que você precisa lembrar

- `String` é usada para representar textos em Java;
- uma `String` é um objeto e possui métodos prontos;
- os índices começam em 0;
- `length()` informa o tamanho;
- `charAt()` acessa um caractere;
- para comparar conteúdo, use `equals()` ou `equalsIgnoreCase()`;
- `Strings` são imutáveis;
- métodos como `substring()`, `replace()`, `contains()` e `indexOf()` facilitam a manipulação de textos;
- `next()` lê uma palavra e `nextLine()` lê uma linha inteira.

Strings aparecem em praticamente todo programa. Dominar seus métodos básicos facilita muito o trabalho com textos em Java.